

Inventario de farmacia con MCP

Proyecto 1: uso de un protocolo existente

Universidad del Valle de Guatemala

21 de septiembre de 2026

Implementación manual · Servidores locales y remotos · Evidencia de red

27 pruebas aprobadas

6 turnos con Gemini real

181 paquetes capturados

13 mensajes JSON-RPC clasificados

Incluye especificación, arquitectura, verificación funcional, análisis por capas, conclusiones y procedimiento de reproducción.

Proyecto 1: uso de un protocolo existente

Universidad del Valle de Guatemala · CC3067 Redes

Caso de estudio: inventario de farmacia mediante Model Context Protocol. **Fecha de verificación:** 21 de septiembre de 2026.

1. Objetivo y resultado

Se implementó un chatbot de consola que accede a Gemini mediante su API, mantiene contexto durante la sesión y coordina tres servidores MCP. Filesystem y Git son servidores oficiales locales. Farmacia es una implementación manual propia disponible tanto por stdio local como por HTTPS en Hetzner.

La demostración real completó seis turnos: pregunta general, seguimiento con contexto, tres consultas de farmacia, seguimiento sobre los resultados, consulta de un SKU inexistente y escritura/commit con los servidores oficiales. Pasaron 27 pruebas automatizadas. Una captura de Wireshark documenta los intercambios remotos completos. Estas evidencias verifican el comportamiento observado, sin sustituir la demostración personal ante el catedrático.

2. Caso de uso y alcance

El caso, aprobado por el catedrático según confirmación del estudiante, permite consultar existencias y detectar productos que requieren reposición. Usa nueve registros ficticios en `data/pharmacy_inventory.json`. No modifica inventario, realiza ventas, interpreta síntomas ni recomienda tratamientos.

La función del LLM es interpretar la solicitud y redactar una respuesta con los datos obtenidos. El anfitrión ejecuta las herramientas. Esta separación permite mostrar de dónde provienen cantidades concretas como las 48 cajas de FAR-001.

3. Arquitectura e implementación

```
Usuario -> CLI -> ChatbotCore -> Gemini API
      |
      +-> McpCoordinator -> cliente stdio -> Filesystem oficial
      +-> McpCoordinator -> cliente stdio -> Git oficial
      +-> McpCoordinator -> cliente stdio -> Farmacia local
                        o cliente HTTPS -> Nginx -> Farmacia remota
```

`ConversationSession` conserva mensajes, llamadas de función, resultados y respuestas solo en memoria. `McpCoordinator` descubre herramientas, expone sus esquemas a Gemini con prefijos por servidor y traduce sus llamadas a JSON-RPC. El log registra solicitudes y respuestas reales con servidor, transporte, método e ID; `/mcp-log` permite consultarlo en la consola.

Los clientes `stdio_client.py` y `http_client.py`, el dispatcher `server_core.py` y ambos transportes se implementaron directamente con JSON y la biblioteca estándar de Python. No se utiliza un SDK MCP ni FastMCP en el código propio. El SDK de Google se utiliza solo para la API del LLM. Los servidores oficiales son procesos externos instalados mediante `npx` y `uvx`.

Las consultas locales y remotas comparten `InventoryService` y `PharmacyMcpServer`; una prueba compara sus resultados. El directorio `demo_workspace/git_demo` está aislado del repositorio principal. El anfitrión lo inicializa y el chatbot usa las herramientas oficiales para escribir, agregar archivos, confirmar cambios y consultar el historial.

4. Especificación del servidor desarrollado

Propiedad	Especificación
Identidad	pharmacy-inventory, versión 0.2.0
Protocolo	JSON-RPC 2.0; MCP 2025-11-25
Capacidad anunciada	tools, listChanged=false
Endpoint local	python -m src.custom_mcp.stdio_server, stdin/stdout UTF-8
Endpoint remoto	https://mcp.canchonfc.online/mcp
Métodos HTTP	POST: mensajes; DELETE: cierre; GET: 405 para SSE opcional
Salud	GET /healthz, HTTP 200
Acceso remoto	HTTPS y token Bearer privado
Sesión remota	MCP-Session-Id, 30 minutos de inactividad, máximo 128

Herramienta	Parámetros	Resultado
get_medication_stock	sku: string obligatorio	Producto con sku, name, stock y unit
search_medications	query: string obligatorio	Objeto items con coincidencias en nombre o SKU
list_low_stock	threshold: entero >=0, opcional, predeterminado 10	Objeto items con stock <= threshold

Los argumentos adicionales se rechazan. Las búsquedas y SKU ignoran mayúsculas; se eliminan espacios externos. Los resultados incluyen un bloque de texto JSON, `structuredContent` equivalente e `isError`. Un SKU desconocido produce `isError:true`; una herramienta desconocida produce error JSON-RPC -32602. También se validan JSON estricto, IDs, inicialización, parámetros y tamaño.

El cliente envía `initialize`, recibe versión/capacidades, emite `notifications/initialized`, descubre con `tools/list` y llama con `tools/call`. Las notificaciones no tienen ID. Las respuestas repiten el ID de su solicitud. La especificación completa con ejemplos reproducibles está en [docs/pharmacy-mcp.md](#); HTTP, cabeceras, estados y operación en [docs/deployment.md](#). Referencias normativas: [MCP 2025-11-25](#), [JSON-RPC 2.0](#).

5. Despliegue sin afectar servicios existentes

Se inspeccionaron los puertos, contenedores, redes, volúmenes y Nginx del host. Se creó exclusivamente el proyecto `Compose pharmacy-mcp` bajo `/home/gil/pharmacy-mcp`, con su propia red y sin volúmenes compartidos. Nginx recibe HTTPS y reenvía hacia `127.0.0.1:8091`; el contenedor escucha en 8000. No se expone directamente ese puerto en Internet.

El contenedor ejecuta Unicorn con un worker y cuatro hilos. Tiene límite de 256 MiB y 0.5 CPU, usuario sin privilegios, filesystem de solo lectura y directorio temporal limitado. El certificado de Let's Encrypt corresponde únicamente al subdominio nuevo. Su renovación tiene una tarea programada y un hook de recarga de Nginx. La clave de Gemini permanece en el cliente.

Después del despliegue, frontend, backend, PostgreSQL y Lavalink conservaron sus identificadores y tiempos de actividad. El bot permaneció activo y el portal existente respondió HTTP 200. El contenedor nuevo se observó saludable y consumió aproximadamente 33.75 MiB en una medición en reposo; no es una prueba de carga.

6. Verificación funcional

Comprobación	Evidencia observada
Suite automatizada	27 pruebas aprobadas; stdio, HTTP y modelo simulado
API y contexto reales	Respuesta sobre Alan Turing y fecha de nacimiento en el siguiente turno
Stock remoto	FAR-001 devuelve 48 cajas
Búsqueda remota	FAR-004: 6 botellas; FAR-005: 12 botellas
Existencias bajas	FAR-002, FAR-004, FAR-006 y FAR-008 con umbral 10
Contexto sobre resultados	Identifica FAR-004 como vitamina con stock <=10
Error de negocio	FAR-999 no existe; el chatbot lo comunica
MCP oficiales	Filesystem escribe README; Git agrega, confirma y muestra el commit
Captura	181 paquetes, 13 mensajes JSON-RPC, 8 conexiones TCP

El commit de la demostración aislada fue `1c2cbf5d0ee49f00fa4ded2e049118be4b9416ea`. El diálogo exacto está en `docs/evidence/live-demo.json`; los 31 mensajes MCP de los tres servidores en `docs/evidence/mcp-exchanges.json`. Los metadatos de los paquetes y los cuerpos descifrados seleccionados están en `docs/evidence/`. Los archivos originales de captura y claves TLS se conservan localmente fuera de Git.

7. Análisis por capas

El anexo de red contiene la tabla de todos los intercambios y su reproducción. En enlace se observan tramas Ethernet II de la LAN; en red, IPv4 entre 192.168.1.152 y 46.225.13.27; en transporte, conexiones TCP a 443; por encima, TLS 1.3 protege HTTP/1.1 con JSON-RPC/MCP. Las direcciones MAC son del enlace local y el TTL no demuestra por sí solo una ruta completa.

Se identificaron la sincronización (frames 14/18 y 37/41), el descubrimiento (60/65), tres llamadas correctas (83/87, 105/109, 128/132), un error de inventario (151/156) y el cierre HTTP (172/176). HTTP 202 confirma una notificación sin introducir una respuesta JSON-RPC. HTTP 200 puede transportar `isError:true`: el éxito del transporte no equivale al éxito de la operación de inventario.

8. Dificultades, límites y lecciones

La implementación manual exigió separar validación JSON-RPC, secuencia MCP y errores de herramientas. Compartir el dispatcher evitó diferencias entre local y remoto. Las pruebas reales sobre sockets y procesos detectan problemas que una prueba de funciones aisladas no cubre.

El tiempo de espera inicial de 15 segundos para el LLM fue insuficiente en una demostración anterior; se ajustó a 60 segundos con un reintento breve para fallos transitorios. El programa comunica errores de disponibilidad y cuota; las pruebas automatizadas no garantizan disponibilidad futura de Gemini.

HTTPS impedía leer JSON directamente desde los paquetes. Exportar secretos TLS solo del cliente de farmacia permitió analizar la sesión sin registrar la clave de Gemini. Después se rotó el token de servicio y se verificó el rechazo del anterior. El log de aplicación facilita depurar llamadas; la captura aporta evidencia independiente de TCP, TLS y HTTP.

El despliegue en un host compartido requirió reservar un puerto y un virtual host propios, validar Nginx antes de recargar y comprobar los servicios previos. No se realizaron reinicios generales de Docker ni migraciones de contenedores.

Límites: inventario estático, contexto no persistente, un worker para sesiones en memoria, autenticación estática sin OAuth, sin SSE persistente y sin interfaz gráfica opcional. Reutilizar conexiones, añadir almacenamiento transaccional y autorización por usuario serían ampliaciones futuras, fuera del alcance probado.

9. Conclusiones

La misma interfaz MCP permitió consultar el inventario local y remoto sin cambiar las herramientas expuestas al modelo. Separar anfitrión, cliente, transporte y lógica de negocio fue esencial para conservar ese comportamiento.

Los resultados verificables provienen del servidor y quedan vinculados mediante IDs a sus solicitudes. El seguimiento conversacional pudo utilizar esos datos en turnos posteriores. La captura confirmó el ciclo de vida y mostró que una sesión MCP puede abarcar varias conexiones TCP.

El proyecto demuestra integración funcional y una metodología reproducible: pruebas automáticas, ejecución con API real, captura y análisis por capas. La evidencia respalda esta ejecución concreta; no representa una certificación de rendimiento, disponibilidad continua ni conformidad con todas las capacidades opcionales del protocolo.

10. Preparación de la entrega

El README en inglés documenta instalación y uso. Se incluye guía de presentación y demostración en docs/presentation.md. Antes de la entrega académica se debe confirmar que GitHub está privado, conceder acceso a docentes/auxiliares y completar los datos de identificación exigidos por el curso. La exposición debe realizarla el estudiante. La interfaz gráfica corresponde a un extra opcional.

Análisis de la comunicación remota

Captura real y alcance

El 21 de septiembre de 2026 se ejecutó el chatbot con Gemini y farmacia remota. Se capturó su tráfico con Dumpcap 4.6.8 y se analizó con TShark 4.6.8, componentes de Wireshark. El archivo es `logs/captures/pharmacy-remote.pcapng`. El inicio del primer paquete fue 06:51:04.809 UTC, equivalente a 00:51:04.809 en Guatemala.

Dato medido	Resultado
Interfaz	Ethernet 3, cliente Windows
Filtro de captura	host 46.225.13.27 and tcp port 443
Paquetes capturados / descartados	181 / 0, según Dumpcap
Tiempo entre primer y último paquete	107.238 s
Ventana configurada de captura	180 s; incluye tiempo sin tráfico
Conexiones TCP	8
Mensajes HTTP reconstruidos	16: 8 solicitudes y 8 respuestas
Mensajes JSON-RPC	13: 6 solicitudes, 6 respuestas y 1 notificación
Retransmisiones detectadas por TShark	0 en esta muestra
TLS negociado	TLS 1.3; suite TLS_AES_256_GCM_SHA384

SHA-256 del archivo original:

```
54ee3f9f6a1358de3e0c288159b3500feb34f1aa6d882750c2a46f11107372b4
```

El filtro incluye únicamente el tráfico HTTPS hacia la IP de Hetzner. No captura la API de Gemini, SSH, DNS ni stdio. Los MCP oficiales locales se comprueban en `docs/evidence/mcp-exchanges.json`; stdio no produce paquetes de red. La captura observa el tramo cliente-Nginx, no el tramo interno Nginx-contenedor.

El cliente exportó claves TLS únicamente desde su contexto SSL de farmacia mediante `MCP_TLS_KEYLOG_FILE`. TShark reconstruyó los flujos TCP y descifró TLS con ese archivo. Sin las claves se observa tráfico cifrado, no JSON en claro. La opción de Wireshark utilizada es `tls.keylog_file`. [Documentación de descifrado TLS](#).

Clasificación de todos los intercambios

Los números corresponden a los frames donde TShark presenta el mensaje HTTP completo después de reconstruirlo. Un mensaje puede ocupar varios segmentos TCP; por eso un frame HTTP no equivale a un único paquete original de aplicación.

Frames cliente / servidor	ID JSON-RPC	Clasificación y operación	Resultado
14 / 18	1	Sincronización: initialize y respuesta	HTTP 200; versión 2025-11-25, capacidades y serverInfo
37 / 41	Sin ID	Sincronización: notifications/initialized	HTTP 202 vacío; no hay respuesta JSON-RPC
60 / 65	2	Solicitud y respuesta: tools/list	HTTP 200; tres herramientas
83 / 87	3	Solicitud y respuesta: get_medication_stock, FAR-001	HTTP 200; 48 cajas; isError=false
105 / 109	4	Solicitud y respuesta: search_medications, vitamin	HTTP 200; FAR-004: 6, FAR-005: 12
128 / 132	5	Solicitud y respuesta: list_low_stock, threshold=10	HTTP 200; FAR-002, FAR-004, FAR-006 y FAR-008
151 / 156	6	Solicitud y respuesta: get_medication_stock, FAR-999	HTTP 200; isError=true, SKU inexistente
172 / 176	No aplica	Cierre de sesión mediante HTTP DELETE	HTTP 204 vacío; no es un mensaje JSON-RPC

initialize también es una solicitud JSON-RPC con su respuesta; se clasifica como sincronización por su función en el ciclo de vida. El ACK de TCP, el HTTP 202 y una respuesta JSON-RPC son mecanismos diferentes. La notificación no lleva id y no tiene respuesta JSON-RPC, aunque HTTP confirme su recepción. Las seis respuestas con ID coinciden con sus solicitudes. El error de inventario no es un fallo de transporte: viaja con HTTP 200 y result.isError=true.

Ejemplo extraído del frame 83:

```
{"jsonrpc": "2.0", "id": 3, "method": "tools/call", "params": {"name": "get_medication_stock", "arguments": {"sku": "FAR-001"}}}
```

La respuesta del frame 87 repite id:3, incluye stock:48 en result.structuredContent y isError:false. Los cuerpos completos están en docs/evidence/network-summary.json, sin cabeceras de autorización ni claves.

Capa de enlace

La interfaz presenta tramas Ethernet II con EtherType 0x0800, que indica IPv4. Las direcciones MAC corresponden al enlace local del cliente y su siguiente salto; no identifican directamente la tarjeta de red del servidor en Alemania. La captura desde Windows no muestra los enlaces de los routers intermedios.

Los frames 1 y 2 tienen 66 bytes y el frame 3 tiene 54 bytes. El frame 4 presenta 1581 bytes en la captura del host. Ese tamaño no permite deducir por sí solo la MTU física: funciones de segmentación/agrupación de la interfaz pueden modificar lo observado por una captura local. No se capturó ARP porque el filtro lo excluye.

Capa de red

El frame 1 sale de 192.168.1.152 hacia 46.225.13.27; el frame 2 regresa en sentido inverso. El TTL observado es 128 en el envío y 50 en la respuesta. No se conoce el TTL inicial remoto a partir de este punto de captura, por lo que no se afirma un número exacto de saltos.

La dirección privada identifica al cliente en su LAN. El acceso a la IP pública atraviesa la red del proveedor y, normalmente, traducción de direcciones; esta captura anterior a esa traducción no permite reconstruir toda la ruta ni ver la dirección pública asignada al cliente. La resolución DNS se comprobó aparte; no se atribuye una consulta DNS a este archivo.

Capa de transporte

La primera conexión usa el puerto efímero 65250 del cliente y el puerto 443 del servidor. Los frames 1, 2 y 3 muestran SYN, SYN-ACK y ACK. Los números relativos de secuencia pasan de 0 a 1 porque SYN consume una posición. El frame 4 transporta 1527 bytes TCP y el frame 5 los confirma con ACK relativo 1528.

TCP ofrece el flujo de bytes que permite reconstruir TLS y HTTP. La sesión MCP se mantiene mediante MCP-Session-Id, no mediante una sola conexión TCP: urllib abre aquí ocho conexiones, una por operación HTTP. Esto simplifica el cliente, pero repite los establecimientos TCP/TLS. No se detectaron retransmisiones en esta muestra; el resultado no garantiza ausencia de pérdidas en otras sesiones.

TLS y capa de aplicación

El ServerHello del frame 6 y los otros siete flujos negocian versión 0x0304 (TLS 1.3) y suite 0x1302 (AES-256-GCM/SHA-384). TLS autentica al servidor y protege los bytes HTTP sobre TCP. El cliente verifica la cadena y el nombre del certificado; el certificado privado del servidor no se copió al cliente. [RFC 8446](#), [TLS 1.3](#).

Sobre TLS se observa HTTP/1.1; los POST contienen JSON-RPC 2.0 y métodos MCP. initialize negocia versión y capacidades, tools/list descubre esquemas, tools/call consulta el inventario y el anfitrión entrega esos resultados al LLM. El transporte local representa los mismos mensajes como líneas UTF-8 por stdio.

Entre solicitud HTTP completa y respuesta completa transcurren aproximadamente 157 a 165 ms en los intercambios registrados. Es una medición desde el cliente que incluye red, proxy y aplicación; no es tiempo puro de ejecución del servidor. Las pausas entre consultas también incluyen interacción con Gemini y otros turnos. No deben interpretarse como latencia de una herramienta MCP.

Reproducción y revisión en Wireshark

Desde la raíz del proyecto, con `.env` en modo HTTP, abre dos terminales. En la primera enumera interfaces y selecciona la que corresponda a tu conexión:

```
& 'C:/Program Files/Wireshark/dumpcap.exe' -D
New-Item -ItemType Directory -Force logs/captures
& 'C:/Program Files/Wireshark/dumpcap.exe' -i 1 `
  -f 'host 46.225.13.27 and tcp port 443' -a duration:180 `
  -w logs/captures/pharmacy-remote.pcapng
```

En la segunda, mientras la captura está activa:

```
. ./venv/Scripts/Activate.ps1
$env:MCP_TLS_KEYLOG_FILE = 'logs/captures/mcp-tls.keys'
python -m scripts.live_demo
Remove-Item Env:MCP_TLS_KEYLOG_FILE
```

Espera a que Dumpcap cierre el archivo antes del análisis. Si Gemini tarda más de tres minutos, amplía la ventana; la captura debe incluir initialize y DELETE. Conserva la evidencia original usando otro nombre para nuevas ejecuciones.

```
python -m scripts.analyze_capture
```

Abre el `.pcapng` en Wireshark. En Edit > Preferences > Protocols > TLS, selecciona `logs/captures/mcp-tls.keys` en (Pre)-Master-Secret log filename. Aplica http para ver los 16 intercambios, `tcp.stream == 0` para la primera conexión o `frame.number == 83 || frame.number == 87` para la consulta FAR-001. En Follow > TLS Stream se puede revisar el contenido reconstruido. Los números cambiarán en otra ejecución. [Manual de Dumpcap](#), [manual de TShark](#).

Los originales y las claves permanecen fuera de Git. El token usado durante esta captura fue revocado; se comprobó que es rechazado y que el nuevo funciona. La evidencia publicada contiene solo metadatos seleccionados y cuerpos JSON-RPC. Antes de compartir otra captura descifrada, cambia también su token de servicio.